

PumpScript

Example Programs & Test Report — Part 1

CSE 341 · Concepts of Programming Languages

Gebze Technical University · 8 May 2026

Student Name: Yusuf Eren Nalbant
Student No: 220104004049
Language: PumpScript — A DSL for Workout Routines
Implementation: Python 3 (lexer.py, parser.py, ast_nodes.py)
Run command: python3 pumpscript.py <file.pump> [-dump-ast]

Overview

This report documents three valid PumpScript programs and five malformed programs, all tested using the Part 1 interpreter (lexer + parser). Each valid program exercises a distinct combination of language features; each error program targets a specific failure mode with a clear, line-numbered error message.

Program	Description	<i>routine</i>	<i>exercise</i>	<i>rest</i>	<i>repeat</i>	<i>if/else</i>
valid1.pump	Chest day + conditional repeat	✓	✓	✓	✓	✓
valid2.pump	Leg day, 3 exercises, 2 rests	✓	✓	✓		
valid3.pump	Full body, 3 routines	✓	✓	✓		✓
error1.pump	Missing closing brace	<i>Parse Error — Line 10</i>				
error2.pump	Illegal character @	<i>Lexer Error — Line 4</i>				
error3.pump	Missing type annotation	<i>Parse Error — Line 2</i>				
error4.pump	Unterminated string	<i>Lexer Error — Line 7</i>				
error5.pump	Missing parentheses in call	<i>Parse Error — Line 12</i>				

Valid Programs

Program 1 — Chest Day Routine

valid1.pump

What this program demonstrates

This is the most feature-rich program. It defines a parametrized **routine** with two **exercise** records separated by a **rest** period. At the global level, an **if** condition checks **completed**, and a **repeat** loop calls the routine three times. All four primitive types (**int**, **float**, **string**, **bool**) appear in the exercise fields, and the parameter **sets** is passed through via **Ident** reference — demonstrating static scoping in the AST.

Source Code

```
1 // Valid Program 1: Chest Day Routine
2 // Demonstrates: routine, exercise, rest, repeat, if/else
3
4 routine chest_day(sets: int) {
5     exercise bench_press {
6         sets:      sets
7         reps:      10
8         weight:    80.0
9         group:     "chest"
10        completed: false
11    }
12    rest 60 seconds
13    exercise chest_fly {
14        sets:      sets
15        reps:      12
16        weight:    20.0
17        group:     "chest"
18        completed: false
19    }
20 }
21
22 if completed == false {
23     repeat 3 times {
24         chest_day(4)
25     }
26 }
```

Listing 1: valid1.pump

Parse Result

```
$ python3 pumpscrip.py examples/valid1.pump
[OK] 'examples/valid1.pump' parsed successfully.
```

AST Output

```
Program
  RoutineDecl(chest_day)
    Param(sets: int)
    Block
      ExerciseBlock(bench_press)
        Field(sets)    -> Ident(sets)
        Field(reps)    -> Literal(int, 10)
```

```
Field(weight) -> Literal(float, 80.0)
Field(group)  -> Literal(string, 'chest')
Field(completed) -> Literal(bool, False)
RestStmt(seconds=60)
ExerciseBlock(chest_fly)
Field(sets)    -> Ident(sets)
Field(reps)    -> Literal(int, 12)
Field(weight)  -> Literal(float, 20.0)
Field(group)   -> Literal(string, 'chest')
Field(completed) -> Literal(bool, False)
IfStmt
Condition: BinOp(==) -> Ident(completed), Literal(bool, False)
Then:
  Block
    RepeatStmt(count=3)
      Block
        RoutineCall(chest_day) -> Literal(int, 4)
```

Key AST Observations

- `Field(sets) → Ident(sets)`: The parameter is referenced by name, not copied — confirming static scoping.
- `RestStmt(seconds=60)`: The domain-specific `rest` construct is a first-class statement node, not embedded in an exercise.
- `BinOp(==)`: The condition `completed == false` is correctly parsed as a binary equality expression.
- `RepeatStmt(count=3)`: The `repeat` count is stored as an integer, not an expression — matching the EBNF rule `<repeat_stmt> = "repeat" INT_LIT "times" <block>`.

Program 2 — Leg Day Routine

valid2.pump

What this program demonstrates

Three exercises with two rest periods in a single routine. Each exercise has a different **weight** value, confirming the design decision that weight is declared per-exercise rather than as a shared routine parameter. The routine is called directly from the global scope without a conditional wrapper, showing that `RoutineCall` is valid at the top level.

Source Code

```
1 // Valid Program 2: Leg Day Routine
2 // Demonstrates: multiple exercises, rest periods, float expressions
3
4 routine leg_day(sets: int) {
5     exercise squat {
6         sets:      sets
7         reps:      8
8         weight:    100.0
9         group:     "legs"
10        completed: false
11    }
12    rest 90 seconds
13    exercise lunges {
14        sets:      3
15        reps:      12
16        weight:    0.0
17        group:     "legs"
18        completed: false
19    }
20    rest 60 seconds
21    exercise leg_press {
22        sets:      sets
23        reps:      10
24        weight:    120.0
25        group:     "legs"
26        completed: false
27    }
28 }
29
30 leg_day(4)
```

Listing 2: valid2.pump

Parse Result

```
$ python3 pumpscrip.py examples/valid2.pump
[OK] 'examples/valid2.pump' parsed successfully.
```

AST Output

```
Program
  RoutineDecl(leg_day)
    Param(sets: int)
    Block
      ExerciseBlock(squat)
        Field(sets)    ->  Ident(sets)
```

```
Field( reps )    -> Literal( int, 8 )
Field( weight )  -> Literal( float, 100.0 )
Field( group )   -> Literal( string, 'legs' )
Field( completed ) -> Literal( bool, False )
RestStmt( seconds=90 )
ExerciseBlock( lunges )
  Field( sets )    -> Literal( int, 3 )      (* hardcoded *)
  Field( reps )    -> Literal( int, 12 )
  Field( weight )  -> Literal( float, 0.0 )
  Field( group )   -> Literal( string, 'legs' )
  Field( completed ) -> Literal( bool, False )
RestStmt( seconds=60 )
ExerciseBlock( leg_press )
  Field( sets )    -> Ident( sets )
  Field( reps )    -> Literal( int, 10 )
  Field( weight )  -> Literal( float, 120.0 )
  Field( group )   -> Literal( string, 'legs' )
  Field( completed ) -> Literal( bool, False )
RoutineCall( leg_day ) -> Literal( int, 4 )
```

Key AST Observations

- `lunges.sets` → `Literal(int, 3)`: Unlike `squat` and `leg_press` which inherit the parameter, `lunges` uses a hardcoded value — both patterns are valid.
- Two `RestStmt` nodes appear between the three exercises at the correct positions in the block.
- `RoutineCall(leg_day)` → `Literal(int, 4)` appears as a top-level node inside `Program`, confirming global-scope call support.

Program 3 — Full Body Routine

valid3.pump

What this program demonstrates

Three separate routine declarations (`upper_body`, `lower_body`, `core`) are called sequentially from a single `if` block, with `rest` statements between the calls. This is the most complex program and exercises nearly every construct in the PumpScript grammar simultaneously, demonstrating composability and multi-routine programs.

Source Code

```
1  // Valid Program 3: Full Body Routine
2  // Demonstrates: multiple routines, conditional logic, expressions
3
4  routine upper_body(sets: int) {
5      exercise pull_up {
6          sets: sets    reps: 8    weight: 0.0
7          group: "back"    completed: false
8      }
9      rest 90 seconds
10     exercise shoulder_press {
11         sets: sets    reps: 10    weight: 40.0
12         group: "shoulders"    completed: false
13     }
14 }
15
16 routine lower_body(sets: int) {
17     exercise deadlift {
18         sets: sets    reps: 5    weight: 140.0
19         group: "back"    completed: false
20     }
21     rest 120 seconds
22     exercise calf_raise {
23         sets: 3    reps: 15    weight: 60.0
24         group: "legs"    completed: false
25     }
26 }
27
28 routine core(sets: int) {
29     exercise plank {
30         sets: sets    reps: 1    weight: 0.0
31         group: "core"    completed: false
32     }
33     rest 45 seconds
34     exercise crunch {
35         sets: sets    reps: 20    weight: 0.0
36         group: "core"    completed: false
37     }
38 }
39
40 if completed == false {
41     upper_body(4)
42     rest 120 seconds
43     lower_body(3)
44     rest 90 seconds
45     core(3)
```

46 }

Listing 3: valid3.pump

Parse Result

```
$ python3 pumpscrip.py examples/valid3.pump  
[OK] 'examples/valid3.pump' parsed successfully.
```

AST Output (condensed)

```
Program  
  RoutineDecl(upper_body)  [Param: sets: int]  
    ExerciseBlock(pull_up)      RestStmt(90)  
    ExerciseBlock(shoulder_press)  
  RoutineDecl(lower_body)  [Param: sets: int]  
    ExerciseBlock(deadlift)      RestStmt(120)  
    ExerciseBlock(calf_raise)  
  RoutineDecl(core)        [Param: sets: int]  
    ExerciseBlock(plank)          RestStmt(45)  
    ExerciseBlock(crunch)  
  IfStmt  
    Condition: BinOp(==) -> Ident(completed), Literal(bool, False)  
    Then:  
      Block  
        RoutineCall(upper_body) -> Literal(int, 4)  
        RestStmt(seconds=120)  
        RoutineCall(lower_body) -> Literal(int, 3)  
        RestStmt(seconds=90)  
        RoutineCall(core)        -> Literal(int, 3)
```

Key AST Observations

- Three `RoutineDecl` nodes appear at the `Program` level — all globally visible, confirming static lifetime of routine declarations.
- `RestStmt` nodes appear both *inside* routines and *between* routine calls inside the `if` block, demonstrating that `rest` is valid at any nesting level.
- The `if` block correctly contains five statements: 3 `RoutineCall` nodes interleaved with 2 `RestStmt` nodes.

Malformed Programs — Error Messages

Error 1 — Missing Closing Brace

error1.pump

```
1 // Error 1: Missing closing brace
2 routine chest_day(sets: int) {
3     exercise bench_press {
4         sets:      sets
5         reps:      10
6         weight:    80.0
7         group:     "chest"
8         completed: false
9     }
10 // Missing closing } for routine
```

Listing 4: error1.pump

```
$ python3 pumpscrip.py examples/error1.pump
[Parse Error] Line 10: Expected '}' but found ''
```

Explanation: The parser expected a closing `}` to end the routine block but reached end-of-file. The empty string `''` in the message represents the EOF token.

Error 2 — Illegal Character

error2.pump

```
1 // Error 2: Invalid character @ in source
2 routine leg_day(sets: int) {
3     exercise squat {
4         sets:  @sets      // @ is not a valid token
5         reps:   8
6         weight: 100.0
7         group:  "legs"
8         completed: false
9     }
10 }
```

Listing 5: error2.pump

```
$ python3 pumpscrip.py examples/error2.pump
[Lexer Error] Line 4: Unexpected character '@'
```

Explanation: The `@` character is not part of any PumpScript token category. The **lexer** (not the parser) catches this before any grammar analysis begins.

Error 3 — Missing Type Annotation

error3.pump

```
1 // Error 3: Missing type annotation in parameter
2 routine chest_day(sets) {      // should be: sets: int
3     exercise bench_press {
4         sets:      sets
5         reps:      10
6         weight:    80.0
7         group:     "chest"
8         completed: false
9     }
10 }
```



```
10 }
```

Listing 6: error3.pump

```
$ python3 pumpscrip.py examples/error3.pump
[Parse Error] Line 2: Expected ':' but found ')'
```

Explanation: PumpScript requires every parameter to carry a type annotation (`name : type`). The parser sees `sets` then `)` and expects `:` in between — EBNF rule `<param> = IDENT ":" <type>`.

Error 4 — Unterminated String

error4.pump

```
1 // Error 4: Unterminated string literal
2 routine leg_day(sets: int) {
3     exercise squat {
4         sets:      sets
5         reps:      8
6         weight:    100.0
7         group:     "legs          // missing closing "
8         completed: false
9     }
10 }
```

Listing 7: error4.pump

```
$ python3 pumpscrip.py examples/error4.pump
[Lexer Error] Line 7: Unterminated string literal -- missing closing ''
```

Explanation: The lexer detects that the string `"legs` has no closing quote before the end of the line. Strings cannot span multiple lines in PumpScript (regex: `"[^\"]*"`).

Error 5 — Missing Parentheses in Call

error5.pump

```
1 // Error 5: Routine call missing parentheses
2 routine chest_day(sets: int) {
3     exercise bench_press {
4         sets:      sets
5         reps:      10
6         weight:    80.0
7         group:     "chest"
8         completed: false
9     }
10 }
11
12 chest_day 4      // should be: chest_day(4)
```

Listing 8: error5.pump

```
$ python3 pumpscrip.py examples/error5.pump
[Parse Error] Line 12: Expected '(' but found '4'
```

Explanation: Routine calls in PumpScript always require parentheses — EBNF rule `<routine_call> = IDENT "(" [ <arg_list> ] )"`. The parser sees `chest_day` and expects `(` next, but finds `4`.

Summary

File	Result	Stage	Line	Feature / Error
valid1.pump	PASS	—	—	routine, exercise, rest, repeat, if/else
valid2.pump	PASS	—	—	3 exercises, 2 rest periods, direct call
valid3.pump	PASS	—	—	3 routines, complex if block, interleaved rest
error1.pump	FAIL	Parser	10	Missing closing }
error2.pump	FAIL	Lexer	4	Illegal character @
error3.pump	FAIL	Parser	2	Missing : in parameter
error4.pump	FAIL	Lexer	7	Unterminated string literal
error5.pump	FAIL	Parser	12	Missing (in routine call

All error messages include the exact line number where the problem was detected. Lexer errors are reported before parsing begins; parser errors identify the specific token mismatch. Both error types follow the format:

[Lexer Error] Line N: <description>
 [Parse Error] Line N: Expected '<X>' but found '<Y>'